

LABORATORIO DI INFORMATICA

Modulo M0

Ambiente di lavoro e ripasso di Git

Esercizi

Istituto Tecnico Tecnologico "G. Marconi" — Rovereto

Indirizzo Informatica e Telecomunicazioni, articolazione Informatica

Classe 4Bi — Anno scolastico 2026-2027 · Docente di laboratorio: Andrea Trentini

Modulo M0 — Esercizi

Gli esercizi vanno svolti sulla postazione del laboratorio e consegnati nel repository personale del corso, nella cartella `M0_ambiente/`. Salvo indicazione diversa, ogni esercizio produce almeno un file e almeno un commit con un messaggio conforme alla convenzione vista a lezione. Dove viene richiesto di riportare l'output di un comando, lo si copia dal terminale in un file di testo o Markdown, senza riscriverlo a memoria.

Negli esempi il nome utente è `rossi` e il repository è `lab-info-4bi-rossi`: sostituire ovunque con i propri.

Livello base

Esercizio 1 — Scheda delle versioni della postazione

Verificare che i tre strumenti dell'ambiente siano installati e funzionanti, e registrare le versioni effettive della propria postazione in un file `M0_ambiente/versioni.md`. Il file deve riportare, per ciascuno strumento, il comando usato per la verifica e la riga di risposta ottenuta, oltre al numero della postazione del laboratorio.

Comandi da eseguire:

```
py --version
code --version
git --version
```

Output atteso: un file `versioni.md` contenente tre blocchi di codice, uno per strumento, con il comando e la sua risposta. La versione di Python deve appartenere alla serie 3.12; se così non è, riportare l'esito della verifica e segnalarlo al docente prima di procedere.

Tempo stimato: 15 minuti

Esercizio 2 — Navigazione e percorsi nel terminale

Partendo dalla cartella personale dell'utente, raggiungere con il terminale la cartella dei documenti, che Esplora risorse mostra come *Documenti* ma che sul disco si chiama `Documents`, creare al suo interno la cartella `esercizio-percorsi` con dentro due sottocartelle `dati` e `risultati`, poi spostarsi in `dati` e da lì raggiungere `risultati` usando un percorso relativo, senza mai scrivere un percorso assoluto dopo il primo spostamento.

Trascrivere in `M0_ambiente/percorsi.md` la sequenza completa dei comandi usati e, in fondo, l'output di `Get-Location` eseguito dalla cartella `risultati`.

Output atteso: un file che documenta la sequenza dei comandi e termina con una riga del tipo

```
C:\Users\rossi\Documents\esercizio-percorsi\risultati.
```

Tempo stimato: 20 minuti

Esercizio 3 — Esecuzione di un programma dal terminale

Creare con Visual Studio Code il file `M0_ambiente/orario.py` contenente un programma che stampa una riga di saluto con il proprio nome e il numero della postazione, ed eseguirlo dal terminale integrato dell'editor.

Contenuto di partenza da completare con i propri dati:

```
nome = "Maria Rossi"
postazione = 12
print("Postazione", postazione, "pronta per", nome)
```

Output atteso, sul terminale, dopo il comando `py orario.py`:

```
Postazione 12 pronta per Maria Rossi
```

Consegnare il file sorgente e, in un file `M0_ambiente/esecuzione.md`, il comando usato e l'output ottenuto.

Tempo stimato: 15 minuti

Esercizio 4 — Configurazione dell'identità in Git

Impostare la configurazione globale di Git con il proprio nome e con l'indirizzo di posta dell'account GitHub d'istituto, il ramo iniziale `main` e Visual Studio Code come editor. Verificare poi il risultato rileggendo la configurazione.

Comando di verifica:

```
git config --list --show-origin
```

Output atteso: un file `M0_ambiente/configurazione_git.md` che riporta le quattro impostazioni richieste come compaiono nell'output del comando di verifica, con l'indicazione del file di configurazione da cui provengono. L'indirizzo di posta deve coincidere con quello registrato su GitHub.

Tempo stimato: 15 minuti

Esercizio 5 — Creazione del repository personale e primo commit

Creare su GitHub un repository privato `lab-info-4bi-cognome`, senza file iniziali, clonarlo o collegarlo a una cartella locale, e produrre il primo commit contenente un file `README.md` con il proprio nome, la classe, l'anno scolastico e una riga che descrive lo scopo del repository. Aggiungere il docente come collaboratore.

Comandi di riferimento per il collegamento e l'invio:

```
git remote add origin git@github.com:rossi/lab-info-4bi-rossi.git
git branch -M main
git push -u origin main
```

Output atteso: il repository visibile su GitHub con un commit e il file `README.md`; in locale, `git status` che riporta l'albero di lavoro pulito e il ramo allineato a `origin/main`.

Tempo stimato: 25 minuti

Livello intermedio

Esercizio 6 — Struttura delle cartelle per l'intero anno

Creare nel repository personale la struttura di cartelle che verrà usata per tutte le consegne dell'anno, una per modulo, dalla `M0_ambiente` alla `M8_concorrenza_rete`. Poiché Git non registra le cartelle vuote, inserire in ciascuna cartella ancora priva di contenuti un file segnaposto `.gitkeep` vuoto, e in ciascuna cartella un file `README.md` con il titolo del modulo. Registrare il tutto in un solo commit.

Struttura di riferimento:

```
lab-info-4bi-rossi/
├── .gitignore
├── README.md
├── M0_ambiente/
├── M1_markdown_jupyter/
├── ... fino a M8_concorrenza_rete/
```

Output atteso: `git ls-files` elenca i file segnaposto e i `README.md` di tutte e nove le cartelle di modulo; su GitHub la struttura è navigabile per intero.

Tempo stimato: 25 minuti

Esercizio 7 — File `.gitignore` e verifica delle regole

Scrivere il file `.gitignore` nella radice del repository personale in modo che vengano ignorate la cache dell'interprete Python, gli ambienti virtuali, i file di stato dei notebook e i file

temporanei di Windows. Per verificare che le regole funzionino, creare nella cartella `M0_ambiente` un ambiente virtuale di prova e controllare che Git non lo segnali.

Comandi per la prova:

```
py -3.12 -m venv M0_ambiente\.venv
git status
git check-ignore -v M0_ambiente/.venv/pyvenv.cfg
```

Output atteso: `git status` non elenca la cartella `.venv` fra i file non tracciati; `git check-ignore -v` risponde indicando il file `.gitignore`, il numero della riga e il modello che ha prodotto l'esclusione. Consegnare il `.gitignore` e un file `M0_ambiente/gitignore_verifica.md` con l'output dei due comandi.

Tempo stimato: 25 minuti

Esercizio 8 — Autenticazione verso GitHub e sua verifica

Configurare l'autenticazione verso GitHub scegliendo fra chiave SSH e token personale su HTTPS, e motivare la scelta rispetto alla postazione in uso. Verificare che l'autenticazione funzioni effettuando un `push` di prova e, nel caso della chiave SSH, con il comando di verifica dedicato.

Comando di verifica per SSH:

```
ssh -T git@github.com
```

Output atteso: un file `M0_ambiente/autenticazione.md` che indica il metodo scelto, due o tre righe di motivazione riferite alla postazione, e l'esito della verifica. Nel file non deve comparire alcuna credenziale: niente token, niente contenuto della chiave privata. La chiave pubblica può essere citata solo per il nome del file, non per il contenuto.

Tempo stimato: 30 minuti

Esercizio 9 — Lettura e interpretazione della cronologia

Produrre nel repository personale almeno cinque commit, ciascuno con un messaggio conforme alla convenzione, modificando in momenti diversi il `README.md` principale e i file della cartella `M0_ambiente`. Estrarre poi la cronologia in forma compatta e commentarla.

Comandi da usare:

```
git log --oneline --graph --decorate
git log -5 --pretty=format:"%h %ad %an %s" --date=short
```

Output atteso: un file `M0_ambiente/cronologia.md` che riporta l'output dei due comandi e, sotto, un commento in prosa di otto-dodici righe che spiega che cosa contiene ciascun commit, che cosa indicano le etichette `HEAD`, `main` e `origin/main` nella riga in cui compaiono, e se al momento dell'estrazione il repository locale fosse allineato al remoto.

Tempo stimato: 30 minuti

Esercizio 10 — README del repository personale

Completare il file `README.md` nella radice del repository perché sia utile a chi lo apre senza conoscerlo: intestazione con nome, classe, materia e anno scolastico; una breve descrizione dello scopo del repository; la descrizione della convenzione di denominazione delle cartelle; l'indicazione di dove si trovano le consegne di ciascun modulo; l'ambiente software usato, con le versioni rilevate nell'esercizio 1.

Output atteso: un `README.md` di venticinque-quaranta righe, con titoli di livello coerente, almeno un blocco di codice con l'albero delle cartelle e nessun collegamento interrotto. Il file deve essere leggibile correttamente nella pagina del repository su GitHub.

Tempo stimato: 30 minuti

Livello avanzato

Esercizio 11 — Recupero di file versionati per errore

Simulare un errore frequente e correggerlo. Creare nella cartella `M0_ambiente` una sottocartella `temporanei` con dentro due file, aggiungerla al repository con `git add` e registrarla con un commit; solo dopo aggiungere la cartella al `.gitignore`. Verificare che il `.gitignore` da solo non basti a rimuoverla dal tracciamento e correggere la situazione mantenendo i file sul disco.

Comandi utili per la diagnosi:

```
git status
git ls-files M0_ambiente/temporanei
git check-ignore -v M0_ambiente/temporanei/nota.txt
```

Output atteso: dopo la correzione, `git ls-files M0_ambiente/temporanei` non restituisce alcuna riga, i due file sono ancora presenti sul disco e la cronologia contiene un commit che documenta la rimozione dal versionamento. Consegnare un file `M0_ambiente/recupero.md` con la sequenza dei comandi e la spiegazione in prosa del motivo per cui il `.gitignore` non agisce sui file già tracciati.

Tempo stimato: 35 minuti

Esercizio 12 — Riallineamento dopo una modifica fatta sul remoto

Provocare in modo controllato il rifiuto di un `push` e risolverlo. Modificare il `README.md` direttamente dall'interfaccia web di GitHub e registrare la modifica con un commit sul remoto; senza scaricare la modifica, modificare in locale un file diverso, per esempio `M0_ambiente/versioni.md`, registrare un commit locale e tentare l'invio.

Sequenza da svolgere in locale dopo la modifica sul sito:

```
git add M0_ambiente/versioni.md
git commit -m "Aggiorna la scheda delle versioni della postazione"
git push
git log --oneline --graph --decorate
```

Output atteso: il primo `push` viene rifiutato con un messaggio che segnala la presenza sul remoto di lavoro non presente in locale; dopo il riallineamento il `push` va a buon fine e `git log --oneline --graph --decorate` mostra `main` e `origin/main` sullo stesso commit. Consegnare un file `M0_ambiente/riallineamento.md` con il messaggio di errore ottenuto, i comandi usati per risolvere e la spiegazione in prosa della causa.

Tempo stimato: 40 minuti

Esercizio 13 — Procedura riproducibile di configurazione della postazione

Redigere in `M0_ambiente/procedura_postazione.md` la procedura completa per portare una postazione Windows appena ripristinata allo stato di lavoro raggiunto in questo modulo, in modo che un compagno possa seguirla senza altre informazioni. La procedura deve essere numerata, contenere i comandi esatti in blocchi di codice con l'indicazione del linguaggio, e prevedere per ogni fase un controllo di verifica con l'output atteso. Deve inoltre includere una sezione finale che elenca almeno quattro messaggi di errore fra quelli incontrati durante il modulo, con la causa e il rimedio di ciascuno.

Vincolo: la procedura non deve contenere credenziali, né percorsi che dipendano da un utente specifico. Dove serve il percorso della cartella personale si usi `$HOME`.

Output atteso: un documento Markdown di sessanta-novanta righe, verificato eseguendone i comandi su una cartella di prova diversa da quella usata negli esercizi precedenti, con il repository che riceve il documento in un commit dedicato.

Tempo stimato: 45 minuti

Criteri di valutazione

La valutazione degli esercizi di questo modulo considera cinque dimensioni, con il peso indicato. Il totale è espresso in ventesimi e convertito in decimi per il registro.

Dimensione	Che cosa si valuta	Pun- ti
Correttezza tecnica	I comandi funzionano, gli output riportati corrispondono a quelli reali, le verifiche sono state effettivamente eseguite	6
Uso di Git	Commit di dimensione ragionevole, messaggi conformi alla convenzione, repository allineato al remoto, nessun file rigenerabile versionato	5
Organizzazione del repository	Struttura delle cartelle rispettata, file collocati dove richiesto, <code>.gitignore</code> completo e verificato	3
Documentazione	Chiarezza dei file Markdown consegnati, uso corretto di titoli e blocchi di codice, spiegazioni in prosa comprensibili e pertinenti	4
Sicurezza delle credenziali	Nessun token, chiave privata o password presente nel repository o nella cronologia dei commit	2

La dimensione relativa alla sicurezza delle credenziali ha carattere dirimente: la presenza di una credenziale nel repository o in un commit della cronologia comporta la valutazione non sufficiente dell'intera consegna, indipendentemente dal punteggio ottenuto nelle altre dimensioni, e richiede la revoca immediata della credenziale esposta.

Gli esercizi del livello base sono richiesti a tutti e costituiscono le competenze minime attese del modulo. Il livello intermedio è richiesto per una valutazione pienamente sufficiente. Il livello avanzato concorre alle valutazioni superiori e può essere svolto anche parzialmente, purché ciò che si consegna sia completo e verificato.